

Exploración de conceptos de conversión A/D con Raspberry Pi Pico 2W

Universidad Militar Nueva Granada – Ingeniería en Telecomunicaciones
Miguel Ángel Plazas – Daniel Mateo Alegría Bernate
Docente: José de Jesús Rúgeles Uribe

Resumen - En esta práctica se estudió experimentalmente el proceso de conversión analógico-digital utilizando una Raspberry Pi Pico 2W basada en el microcontrolador RP2350. En la primera parte se analizó el proceso de muestreo de una señal senoidal, determinando la frecuencia y el período de muestreo, así como el número de muestras obtenidas por período para diferentes frecuencias de entrada. En la segunda parte se evaluó estadísticamente el comportamiento del ADC mediante cinco niveles de tensión DC y 10 000 mediciones por ensayo.

Palabras clave - ADC, conversión analógico-digital, muestreo, Raspberry Pi Pico 2W, RP2350, resolución, exactitud, repetibilidad.

Abstract — In this experiment, the analog-to-digital conversion process was studied using a Raspberry Pi Pico 2W based on the RP2350 microcontroller. The first part analyzed the sampling process of a sinusoidal signal, determining the sampling frequency and period, as well as the number of samples obtained per period for various input frequencies. The second part involved a statistical evaluation of the ADC's behavior using five DC voltage levels and 10,000 measurements per trial.

Keywords— ADC, analog-to-digital conversion, sampling, Raspberry Pi Pico 2W, RP2350, resolution, accuracy, repeatability

I. INTRODUCCIÓN

La conversión analógico-digital (A/D) es un proceso fundamental en los sistemas de comunicación digital, ya que permite transformar señales analógicas en valores digitales que pueden ser procesados por sistemas electrónicos. Este proceso se basa en la toma de muestras de una señal continua a determinados intervalos de tiempo, por lo que la frecuencia de muestreo y el número de muestras obtenidas por período son parámetros importantes para representar adecuadamente la señal.

En esta práctica de laboratorio se exploró experimentalmente el proceso de conversión A/D utilizando una Raspberry Pi Pico 2W, basada en el microcontrolador RP2350.

A partir de las mediciones realizadas se determinaron experimentalmente el período y la frecuencia de muestreo, así

como el período y la frecuencia de la señal de entrada. Además, se analizó cómo cambia el número de muestras disponibles por período al aumentar la frecuencia de la señal manteniendo constante la frecuencia de muestreo. De esta manera, se buscó relacionar los resultados experimentales con los conceptos fundamentales de muestreo y conversión analógico-digital.

II. OBJETIVOS

A. Objetivo General

Comprender los fundamentos de la conversión analógico-digital mediante su observación experimental y analizar su aplicación en la adquisición y procesamiento de señales dentro de un sistema de comunicación digital

B. Objetivos Específicos

- Determinar la frecuencia y el período de muestreo implementados en la Raspberry Pi Pico 2W.
- Establecer la relación entre la frecuencia de entrada y el número de muestras por período.
- Caracterizar el comportamiento estadístico del ADC ante diferentes niveles de tensión DC.
- Examinar los códigos digitales obtenidos y su correspondencia con la resolución de 12 bits del convertidor.
- Contrastar el LSB ideal con la desviación estándar para identificar la variabilidad del sistema de adquisición.

III. MARCO TEÓRICO

A. Conversión analógico-digital

La conversión analógico-digital (A/D) es el proceso mediante el cual una señal analógica continua se transforma en una representación digital. Este proceso comprende las etapas de muestreo y cuantización, permitiendo representar los valores de una señal mediante códigos digitales. [1]. La resolución del convertidor determina la cantidad de niveles disponibles para representar la amplitud de la señal [1].

B. Muestreo

El muestreo consiste en obtener valores de una señal continua en intervalos de tiempo determinados. [2]. El período de muestreo T_s corresponde al intervalo entre dos muestras consecutivas y se relaciona con la frecuencia de muestreo F_s mediante:

$$T_s = \frac{1}{F_s}$$

De manera equivalente:

$$F_s = \frac{1}{T_s}$$

C. Raspberry Pi Pico 2W y ADC

La Raspberry Pi Pico 2 W es una placa de desarrollo basada en el microcontrolador RP2350. Este microcontrolador incorpora un convertidor analógico-digital (ADC) de 12 bits, utilizado para convertir señales analógicas en valores digitales. El ADC dispone de entradas analógicas asociadas a determinados pines GPIO, entre ellas GP26, correspondiente al canal ADC0 [1].

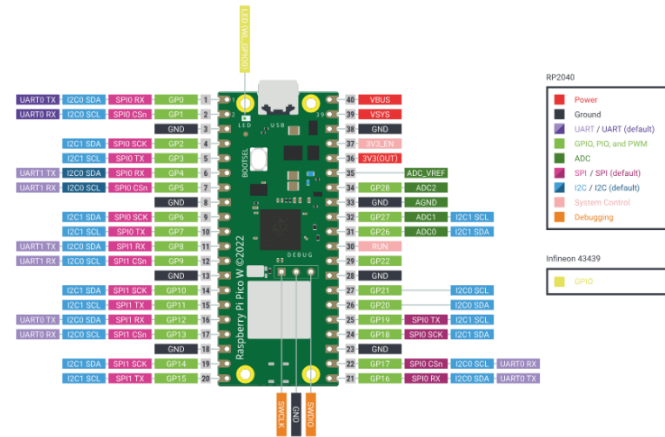


Figure 1. Raspberry Pi Pico 2W

En la Figura 1 se identifican los pines utilizados para la adquisición y visualización del proceso de muestreo. El pin **GP26/ADC0** corresponde a la entrada de la señal analógica proveniente del generador de funciones, mientras que **GND** proporciona la referencia común entre los equipos.

El pin **GP15** corresponde a la salida digital utilizada para indicar los instantes de adquisición. De esta manera, la señal analógica se encuentra asociada a la entrada del ADC y la señal digital permite identificar temporalmente las muestras realizadas.

D. Muestras por período

El número de muestras obtenidas durante un período de una señal depende de la relación entre la frecuencia de muestreo y la frecuencia de la señal:

$$N_s = \frac{F_s}{f_{in}}$$

donde N_s representa el número de muestras por período, F_s la frecuencia de muestreo y f_{in} la frecuencia de la señal.

También puede expresarse mediante los períodos:

$$N_s = \frac{T_{in}}{T_s}$$

Por consiguiente, manteniendo constante la frecuencia de muestreo, un aumento en la frecuencia de la señal produce una disminución del número de muestras disponibles para representar cada período.

E. Teorema de Nyquist

El teorema de Nyquist-Shannon establece que una señal limitada en banda puede ser reconstruida correctamente a partir de sus muestras cuando la frecuencia de muestreo es superior al doble de la máxima frecuencia presente en la señal [2]:

$$F_s = 2 f_{max}$$

A partir de esta condición se define la frecuencia de Nyquist como:

$$f_N = \frac{F_s}{2}$$

F. Resolución del ADC

La resolución de un convertidor analógico-digital determina el número de niveles de cuantización que puede utilizar para representar una señal. Para un ADC de N bits, el número de niveles está dado por:

$$L = 2^N$$

El RP2350 dispone de un ADC con resolución nominal de 12 bits, por lo que:

$$L = 2^{12} = 4096$$

Los códigos digitales se encuentran entre 0 y:

$$2^{12} - 1 = 4095$$

La resolución ideal en tensión, o tamaño de un LSB, puede calcularse mediante:

$$LSB = \frac{V_{REF}}{2^N - 1}$$

Para $V_{REF}=3,3$ V

$$LSB = \frac{3.3 V}{4095} = 0.806 mV$$

Esto significa que, idealmente, cada incremento del código digital representa aproximadamente 0,806 mV dentro del rango de referencia establecido

G. Lectura del ADC en MicroPython

La función “read_u16()” de MicroPython entrega valores entre 0 y 65535. Sin embargo, esto no significa que el ADC posea una resolución física de 16 bits. La conversión realizada por el ADC del RP2350 continúa siendo de 12 bits y posteriormente la lectura es representada mediante una escala de 16 bits [1][3].

La relación indicada para recuperar el código original de 12 bits es:

$$Code_{12} = raw16 \gg 4$$

Donde raw16 corresponde al valor obtenido mediante read_u16().

Finalmente, para convertir una lectura de read_u16() en una tensión aproximada puede utilizarse: [3].

$$V_{in} = \frac{raw16}{65535} \times V_{REF}$$

IV. RESULTADOS EXPERIMENTALES

I. Parte 1

En esta fase se implementó el montaje de adquisición y visualización con la Raspberry Pi Pico 2W conectada a un generador de señales y un osciloscopio digital ejecutando el programa sampling_1.py ver ANEXO C.

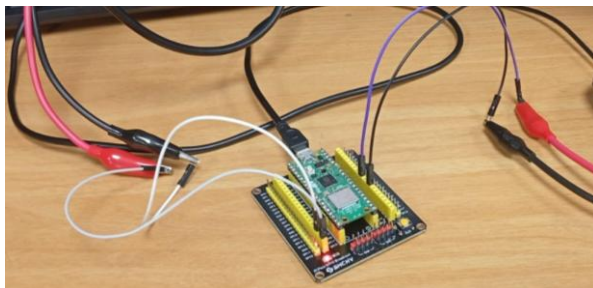


Figure 2. Montaje

Podemos observar como la señal del generador se conecta a la entrada GP26/ADC0 de la Raspberry Pi Pico 2 W y al canal 1 del osciloscopio, mientras que GP15 se conecta al canal 2 para visualizar los instantes de muestreo. Todos los equipos comparten una referencia de tierra común.

A. Preparación de la Señal de Entrada

Inicialmente, se verificó que la señal de entrada se encontrara dentro del rango de tensión permitido por la entrada ADC. Para ello, el generador de funciones se configuró con los siguientes parámetros:

- Señal senoidal
- $f = 100 \text{ Hz}$
- $V_{pp} = 2.0 \text{ V}$
- Offset = 1.65 V.

Antes de conectar el generador a la Raspberry Pi Pico 2W, la señal se observó en el canal 1 (CH1) del osciloscopio para comprobar los valores máximos y mínimos de la señal.

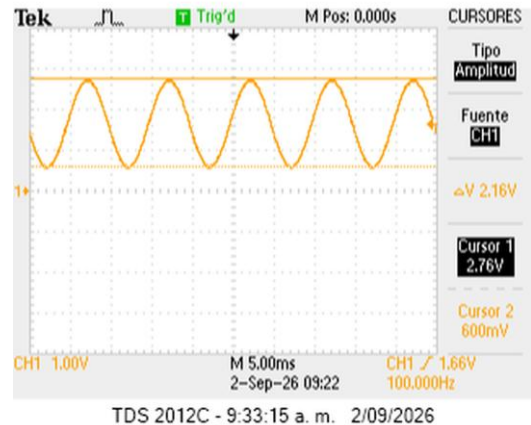


Figure 3. Verificación inicial de la señal senoidal

A partir de esta medición se registraron los valores máximo y mínimo obtenidos permiten verificar que la señal permanece dentro del rango de 0 a 3,3 V requerido por la entrada ADC. Una vez identificado los datos se procedió a comparar los resultados se presentan en la Tabla 1.

Vmax

$$\%E = \frac{|2.76 V - 2.65 V|}{2.65 V} \times 100 = 4,15\%$$

Vmin

$$\%E = \frac{|600 mV - 650 mV|}{650 mV} \times 100 = 7,69\%$$

Tabla 1. Comparación teórico/experimental de Vmax y Vmin

Magnitud	Teórico	Exp	% E
Vmax	2,65 V	2,76 V	4,15%
Vmin	650 mV	600 mV	7,69%

La comparación muestra que el valor máximo experimental fue de 2,76 V, frente a los 2,65 V esperados, lo que corresponde a un error de 4,15 %. Para el valor mínimo se obtuvo 0,60 V, frente a los 0,65 V teóricos, con un error de 7,69 %.

Aunque existen diferencias entre los valores configurados y los medidos, la señal permanece completamente dentro del intervalo de 0 a 3,3 V. Por tanto, las condiciones de amplitud y offset fueron adecuadas para continuar con la conexión al ADC

Las diferencias observadas pueden atribuirse a la tolerancia y precisión de los instrumentos y del generador de funciones, además de las condiciones de medición

B. Medición Experimental de Frecuencia de Muestreo

Al ejecutar el programa `sampling_1.py`, se configuró una frecuencia de muestreo nominal de $F_{(nom)s} = 1000\text{Hz}$, correspondiente a un período entre muestras teórico de $T_{(nom)s} = 1000\mu\text{s}$ (1,0ms).

Se realizaron cinco mediciones para obtener un promedio de los datos a continuación se muestra la primera captura, Para ver todas las imágenes diríjase al ANEXO A

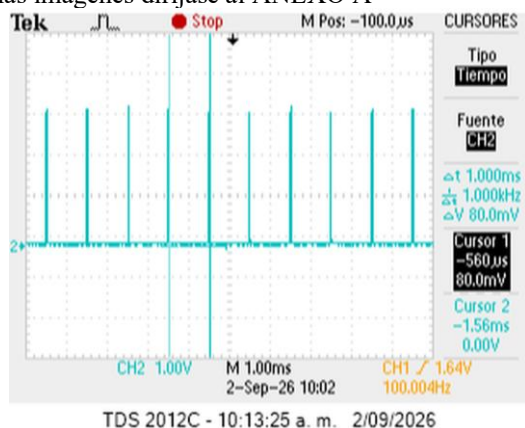


Figure 4 Medición del período de muestreo $T_s(osc)$ con cursores de tiempo sobre CH2: $\Delta t = 1,000\text{ ms}$, $F_s = 1,000\text{ kHz}$.

A través de la medición con cursores en el Canal 2 (CH2) del osciloscopio, pudimos determinar el intervalo entre muestras sucesivas (Δt) y la frecuencia de muestreo real en hardware $F_s^{(osc)}$ que nos proporcionaba el GP15.

Estos valores se compararon con los valores estimados por software a continuación

Tabla 2.Comparación de F_s nominal, Δt , $T_s(osc)$ y $F_s(osc)$

Magnitud	Resultado
$F_s^{(nom)}$	1000Hz
Δt	1ms
$T_s^{(osc)}$	1ms
$F_s^{(osc)}$	1000Hz

El período de muestreo medido con los cursores del osciloscopio (1,000ms) coincide exactamente con el período nominal programado (1 ms), arrojando un error del 0,0% dentro de la resolución de medición del instrumento.

C. Comparación con la frecuencia calculada por software

Al finalizar la adquisición, el programa muestra en la consola una estimación de la frecuencia de muestreo y otros parámetros asociados a los intervalos entre muestras

```

Target Fs           : 1000.00 Hz
Software-estimated Fs : 1000.02 Hz
Mean sample period   : 999.98 us
Minimum period       : 809 us
Maximum period       : 1007 us
  
```

Figure 5.Valores consola

Podemos observar que la frecuencia estimada por software es prácticamente igual a la frecuencia nominal de 1000Hz. Teniendo de igual manera un valor similar con la medición realizada mediante los cursores del osciloscopio.

Una vez identificado los datos se procedió a comparar los resultados

$$\%E = \frac{|1000,2\text{Hz} - 1000\text{Hz}|}{1000\text{Hz}} \times 100 = 0.02\%$$

Tabla 3.Comparación de F_s nominal, software, osciloscopio y %Error

$F_s^{(nom)}$	$F_s^{(soft)}$	$F_s^{(osc)}$	%Error
1000 Hz	1000,02Hz	1000 Hz	0,02 %

La diferencia obtenida entre el valor nominal y la frecuencia estimada por software es de aproximadamente 0,02%, lo que representa una variación muy pequeña y confirma la estabilidad de la frecuencia de muestreo durante la adquisición.

D. Medición de la frecuencia de la señal analógica

A continuación, mediante los cursores del canal 1 (CH1), se midió el período de la señal senoidal de entrada.

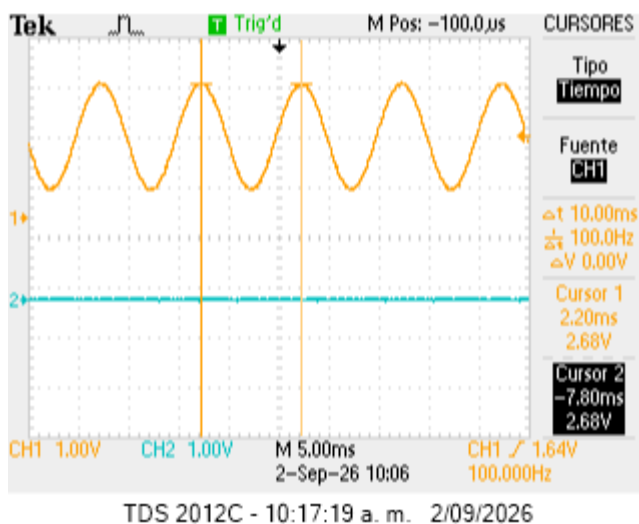


Figure 6. Periodo CH1

De acuerdo con la medición realizada en el osciloscopio, se obtuvo un período de entrada de $T_{in} = 10,0\text{ ms}$. A partir del

período medido, la frecuencia de la señal de entrada se calculó como:

$$f_{in} = \frac{1}{10 \text{ ms}} = 100 \text{ Hz}$$

Con la frecuencia de muestreo y la frecuencia de entrada determinadas experimentalmente, se calculó el número de muestras que toma por período de la señal.

$$N_s = \frac{1000 \text{ Hz}}{100 \text{ Hz}} = 10 \text{ muestras / periodo}$$

Al ejecutar la señal en el osciloscopio pudimos obtener una confirmación con respecto a lo calculado

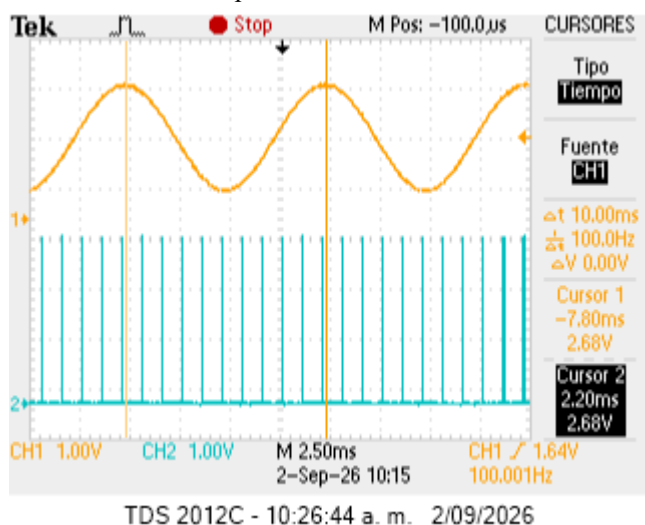


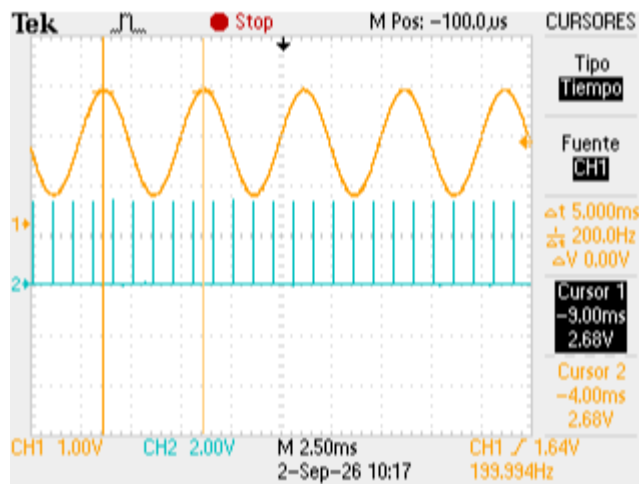
Figure 7. Frecuencia 100 Hz

Podemos observar como el número de flancos ascendentes de CH2 que caen exactamente dentro de un ciclo completo de la onda senoidal en CH1, fue exactamente 10 pulsos, confirmando de la teoría del muestreo.

E. Variación Frecuencia señal de Entrada

Manteniendo fija la frecuencia de muestreo 1000 Hz, se varió progresivamente la frecuencia en el generador de funciones para analizar cómo cambia el número de muestras disponibles por período.

Podemos ver a continuación el ejemplo de la señal visualizada utilizando como ejemplo la primera condición 200 Hz. Para ver las demás configuraciones diríjase a **ANEXO B**



TDS 2012C - 10:28:29 a. m. 2/09/2026

Figure 8 Variación Frecuencia 200 Hz

Podemos observar que, al aumentar la frecuencia de la señal a 200 Hz, disminuye el número de muestras disponibles por período a 5. Esto produce una representación menos densa de la señal.

Tabla 4. Número de muestras por período (N_s) para frecuencias variadas

f_{in} configurada	f_{in} Exp	N_s Calculada	Observaciones
100 Hz	100	10	Excelente definición. Se observan 10 pulsos de muestra
200 Hz	200	5	Señal menos densa y más simétrica en cuanto a los pulsos.
250 Hz	250	4	La densidad de muestreo disminuye.
400 Hz	400	2,5	Señal a límite de Reconstrucción

Este cambio presenta una relación inversa entre la frecuencia de entrada y el número de muestras por período. Al pasar de 100 Hz a 400 Hz, manteniendo $F_s = 1000 \text{ Hz}$, N_s disminuye de 10 a 2,5 muestras por ciclo.

II. Parte II

En esta segunda parte de la práctica se analizó experimentalmente la variabilidad de las mediciones realizadas por el convertidor analógico-digital de la Raspberry Pi Pico 2W al aplicar diferentes niveles de tensión DC aproximadamente constantes.

Para ello, se utilizó la entrada GP26/ADC0 y se realizaron cinco ensayos con diferentes valores de tensión, tomando 10 000 mediciones para cada condición mediante el programa `sampling_2.py`. Puede verse en el **ANEXO D**

A. Montaje experimental y adquisición de las mediciones

Inicialmente, se conectó una tensión DC a la entrada **GP26/ADC0** de la Raspberry Pi Pico 2W. Antes de iniciar cada ensayo, el valor de tensión aplicado fue verificado mediante un multímetro digital, con el objetivo de utilizar esta medición como valor de referencia. Pueden verse en el **ANEXO E**

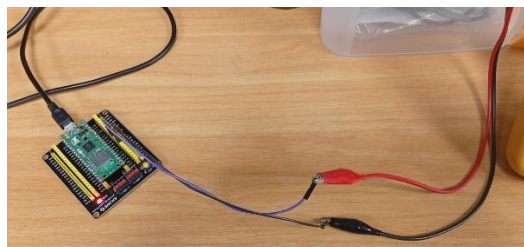


Figure 9. Montaje experimental para las mediciones de tensión DC

Se puede observar la conexión de la fuente de tensión a la entrada **GP26/ADC0**, mientras que el multímetro se utilizó para verificar el valor aplicado a la entrada del ADC. Los valores promedio obtenidos por el ADC y sus respectivas desviaciones estándar se compararon con las tensiones medidas mediante el multímetro. Los resultados obtenidos se presentan en la siguiente tabla.

Tabla 5. Resultados de las mediciones realizadas con el ADC

Test	V_DMM [V]	V_ADC [V]	s [mV]	%E
1	0,599	0,5974	14,451	0,27
2	1,199	1,1978	15,377	0,1
3	1,798	1,8011	15,878	0,17
4	2,399	2,4042	15,542	0,22
5	2,999	3,0061	15,332	0,24

A partir de los resultados se observa que los valores promedio obtenidos por el ADC presentan una diferencia pequeña respecto a las mediciones realizadas con el multímetro. El mayor valor absoluto del error corresponde al **Test 1, con 0,27 %**, mientras que en los demás ensayos el error se mantiene por debajo del **0,25 %**. La desviación estándar presenta valores comprendidos aproximadamente entre **14,45 mV y 15,88 mV**. Esto indica que la dispersión de las mediciones es relativamente similar para los cinco niveles de tensión analizados y que el comportamiento estadístico del ADC no presenta una variación considerable al aumentar la tensión aplicada.

B. Procesamiento de los datos mediante Python

Una vez realizadas las cinco pruebas experimentales, se procesaron los archivos de datos obtenidos mediante el programa `sampling_2.py`. Para este análisis se desarrolló un código en Python encargado de procesar las 10 000 muestras correspondientes a cada ensayo.

El código permite obtener la media y la desviación estándar de las mediciones, además de generar las gráficas necesarias para

analizar el comportamiento de las lecturas del ADC. De esta manera, se facilita la comparación entre los diferentes niveles de tensión aplicados y se puede evaluar la dispersión de las mediciones.

Para ver el código completo diríjase a **ANEXO F**

C. Resultados y análisis

A continuación, se presentan las gráficas obtenidas para los diferentes ensayos. Como ejemplo representativo se presenta inicialmente el **Test 1**. Para ver las gráficas del resto de los ensayos diríjase al **ANEXO G y ANEXO H**.

Para el test 1 pudimos obtener los siguientes resultados

```

Statistical results
-----
N                : 10000
Mean V_bar       : 0.597400 V
Sample variance s^2 : 2.0882466000e-04 V^2
Sample std. deviation s : 0.014451 V
Sample std. deviation s : 14.451 mV
Std. deviation    : 17.932 LSB
Minimum V_min     : 0.458531 V
Maximum V_max     : 0.707535 V
Range R           : 0.249004 V
Minimum nominal code : 569
Maximum nominal code : 878
Mode nominal code   : 739
Mode count          : 449
Mode equivalent voltage : 0.595531 V
Code 0 occurrences   : 0
Code 4095 occurrences : 0

DMM reference      : 0.600000 V
Mean error ADC - DMM : -0.002600 V
Mean error ADC - DMM : -2.600 mV
Relative mean error : -0.433 %
  
```

Figure 10. Resultados estadísticos de consola — Test 1

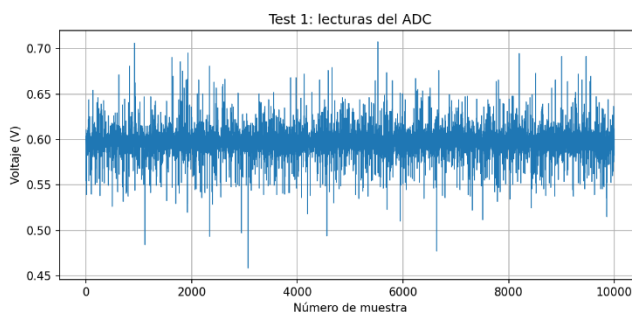


Figure 11. Voltaje vs. número de muestra (Test 1)

En la gráfica se observa que las mediciones se concentran alrededor de un valor cercano a **0,5974 V obteniendo** un valor mínimo de **0,4585 V** y un valor máximo de **0,7075 V**, aunque presentan pequeñas variaciones entre una muestra y otra. Estas variaciones corresponden a la dispersión propia de las mediciones del ADC y permiten observar que, aunque la tensión aplicada permanece aproximadamente constante, las lecturas individuales no son exactamente iguales.

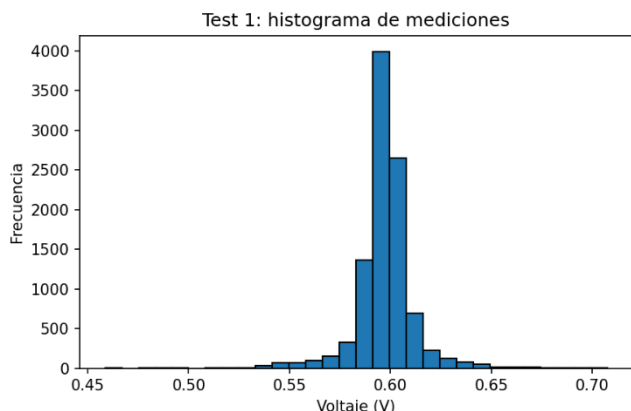


Figure 12. Histograma de las mediciones en voltios para el Test 1.

El histograma permite observar la distribución de las 10 000 mediciones alrededor del valor promedio. De esta manera, a diferencia de una única lectura, es posible identificar la concentración de los datos y la dispersión existente alrededor de la tensión aplicada.

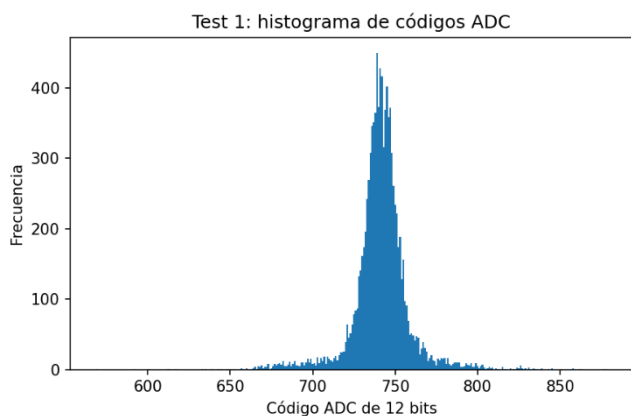


Figure 13. Histograma de códigos ADC de 12 bits (Test 1)

Este histograma permite identificar los diferentes códigos digitales generados por el ADC para una tensión DC aproximadamente constante. La presencia de varios códigos alrededor del valor central demuestra que pequeñas variaciones en la tensión medida se reflejan en cambios en el código digital entregado por el convertidor.

V. ANÁLISIS Y DISCUSIÓN DE RESULTADOS

A. Proceso de muestreo

Durante la primera parte se comprobó que la frecuencia de muestreo implementada fue cercana a la establecida de $F_s=1000$ Hz, obteniendo un período de muestreo aproximado de $T_s=1$ ms. Al aumentar la frecuencia de entrada a 200, 250 y 400 Hz, el número de muestras por período disminuyó a aproximadamente 5, 4 y 2,5, respectivamente. Esto permitió comprobar experimentalmente que, manteniendo constante la frecuencia de muestreo, una mayor frecuencia de entrada produce menos muestras por período.

B. Análisis de la dispersión de las mediciones

La desviación estándar permite determinar qué tan dispersas se encuentran las mediciones alrededor de su valor promedio. En los cinco ensayos se obtuvieron valores comprendidos entre 14,451 mV y 15,878 mV. El menor valor de desviación estándar se presentó en el Test 1, con 14,451 mV, mientras que el mayor correspondió al Test 3, con 15,878 mV. La diferencia entre ambos valores es de:

$$15,878 - 14,451 = 1,427 \text{ mV}$$

Aunque existe una pequeña variación entre los ensayos, los valores de desviación estándar permanecen dentro de un intervalo relativamente reducido. Además, no se observa que la desviación estándar aumente de manera proporcional con la tensión aplicada, ya que después de alcanzar el máximo en el Test 3, disminuye ligeramente en los Tests 4 y 5.

C. Análisis de los histogramas y de los códigos ADC

Los histogramas obtenidos a partir de las 10000 mediciones permiten observar la distribución de los valores registrados por el ADC. Aunque la tensión aplicada a la entrada se mantuvo aproximadamente constante en cada ensayo, las mediciones no produjeron exactamente el mismo valor en todas las muestras. En cambio, los valores se distribuyeron alrededor de una media. Esto también puede observarse al representar la tensión medida en función del número de muestra, donde las mediciones presentan pequeñas variaciones alrededor del valor promedio. Por esta razón, el histograma permite visualizar de una manera más clara la dispersión presente en cada ensayo.

D. Análisis códigos ADC

Además, los histogramas de los códigos digitales muestran que una tensión DC aproximadamente constante puede generar diferentes códigos del ADC. Esto ocurre debido a las pequeñas variaciones presentes en las mediciones, haciendo que algunas muestras sean convertidas a un nivel de cuantización diferente. El RP2350 utiliza un ADC de resolución nominal de 12 bits, por lo que dispone de

$$2^{12} = 4096$$

niveles de cuantización, correspondientes a códigos entre 0 y 4095. Por lo tanto, aunque la entrada sea una tensión DC constante, las variaciones de la medición pueden provocar que se obtengan varios códigos digitales alrededor del valor central.

E. Comparación con el LSB ideal del ADC

Debido a que el ADC posee una resolución nominal de 12 bits y se considera una tensión de referencia de 3,3 V, el tamaño ideal de un nivel de cuantización puede calcularse mediante:

$$LSB = \frac{V_{REF}}{2^{12} - 1}$$

$$LSB = \frac{3,3 V}{4095} = 0,806 mV$$

Este valor representa el incremento ideal de tensión asociado con un cambio de un código del ADC al siguiente.

Al comparar este valor con las desviaciones estándar experimentales, se observa que la dispersión obtenida es considerablemente mayor que un LSB.

Por ejemplo, en el Test 1:

$$LSB = \frac{14,451}{0,806} = 17,93$$

mientras que en el Test 3:

$$LSB = \frac{15,878}{0,806} = 19,70$$

F. Diferencia entre las mediciones del ADC y el multímetro

Otra forma de analizar los resultados consiste en calcular directamente la diferencia entre la tensión promedio obtenida mediante el ADC y la tensión medida con el multímetro:

$$\Delta V = V_{ADC} - V_{DMM}$$

Para los cinco ensayos se obtuvieron aproximadamente los siguientes valores:

- Test 1: $\Delta V = -1,6 mV$
- Test 2: $\Delta V = -1,2 mV$
- Test 3: $\Delta V = +3,1 mV$
- Test 4: $\Delta V = +5,2 mV$
- Test 5: $\Delta V = +7,1 mV$

Se observa que la diferencia aumenta ligeramente al incrementar el nivel de tensión. A pesar de esto, los valores promedio obtenidos mediante el ADC permanecen próximos a las mediciones realizadas con el multímetro, lo que indica una buena correspondencia entre ambos instrumentos.

G. Exactitud y repetibilidad

La exactitud se evaluó comparando la media del ADC con el valor medido mediante el multímetro. Los errores inferiores al 0,3 % muestran que los valores promedio presentan una buena cercanía al valor de referencia. La repetibilidad, por su parte, se analizó mediante la desviación estándar y los histogramas. Los valores obtenidos muestran una dispersión relativamente similar entre los cinco ensayos. Por lo tanto, ambos conceptos son diferentes: la exactitud indica qué tan cerca está la medición del valor de referencia, mientras que la repetibilidad indica qué tan similares son las mediciones entre sí.

VI. CONCLUSIONES

A. Se comprobó experimentalmente el funcionamiento del proceso de muestreo implementado en la Raspberry Pi Pico 2W, obteniendo una frecuencia de muestreo cercana a los 1000 Hz y un período aproximado de 1 ms. La comparación entre los valores teóricos y los obtenidos mostró una correspondencia adecuada, confirmando que el sistema de adquisición trabajó bajo las condiciones establecidas inicialmente.

B. Se verificó la relación entre la frecuencia de la señal de entrada y el número de muestras obtenidas por período. Para una señal de 100 Hz se obtuvieron aproximadamente 10 muestras por período, mientras que al aumentar la frecuencia hasta 200, 250 y 400 Hz, este número disminuyó progresivamente determinado así que manteniendo constante la frecuencia de muestreo, el número de muestras disponibles para representar una señal disminuye a medida que aumenta su frecuencia.

C. El análisis estadístico de los cinco niveles de tensión DC permitió comprobar que el valor promedio obtenido por el ADC se mantuvo cercano a la medición realizada con el multímetro. Los errores porcentuales estuvieron entre -0,27 % y +0,24 %, lo que representa una diferencia inferior al 0,3 % en todos los casos. Esto evidencia una buena correspondencia entre el valor promedio proporcionado por el sistema de adquisición y el valor utilizado como referencia.

D. Se analizó el comportamiento de los códigos generados por el ADC considerando su resolución nominal de 12 bits, correspondiente a 4096 niveles de cuantización y códigos entre 0 y 4095. La presencia de diferentes códigos para una misma tensión DC se relaciona con las variaciones presentes durante la adquisición, demostrando que una entrada aproximadamente constante puede generar pequeñas variaciones en el resultado digital.

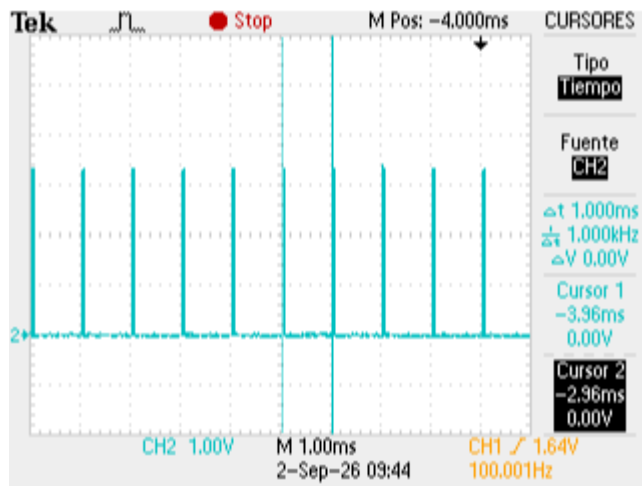
E. Se calculó un LSB ideal de aproximadamente 0,806 mV para una referencia de 3,3 V y una resolución de 12 bits. Al compararlo con las desviaciones estándar obtenidas experimentalmente, se encontró que estas fueron aproximadamente entre 18 y 20 veces mayores que un LSB. Esta comparación permitió diferenciar entre la resolución ideal del ADC y la variabilidad de una medición real, mostrando que la resolución nominal no es suficiente por sí sola para caracterizar el comportamiento del sistema de adquisición.

VII. REFERENCIA

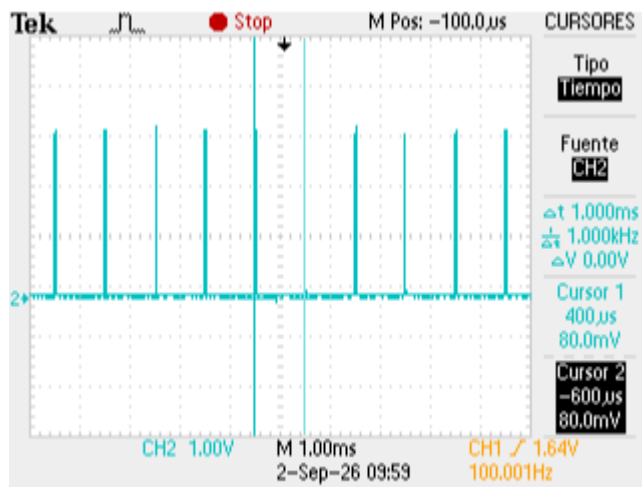
- [1] Raspberry Pi Ltd., Raspberry Pi Pico 2 Datasheet, Raspberry Pi Ltd., 2025. Raspberry Pi Pico 2 Datasheet
- [2] Raspberry Pi Ltd., RP2350 Datasheet, Raspberry Pi Ltd., 2025. RP2350 Datasheet
- [3] Raspberry Pi Ltd., Raspberry Pi Pico-series Python SDK, Raspberry Pi Ltd. Raspberry Pi Pico-series Python SDK
- [4] MicroPython, Quick reference for the RP2, MicroPython Documentation. MicroPython – Quick reference for RP2
- [5] Raspberry Pi Ltd., Raspberry Pi Documentation: Microcontroller chips – RP2350, Raspberry Pi Ltd. Documentación oficial del RP2350

VIII. ANEXOS

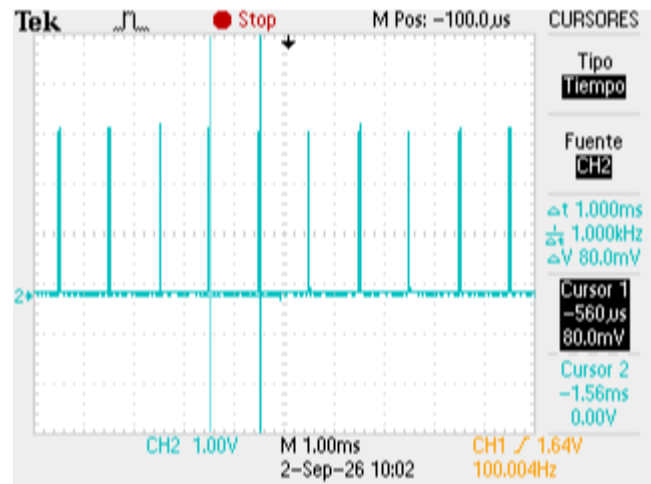
A. ANEXO A Periodo Promedio



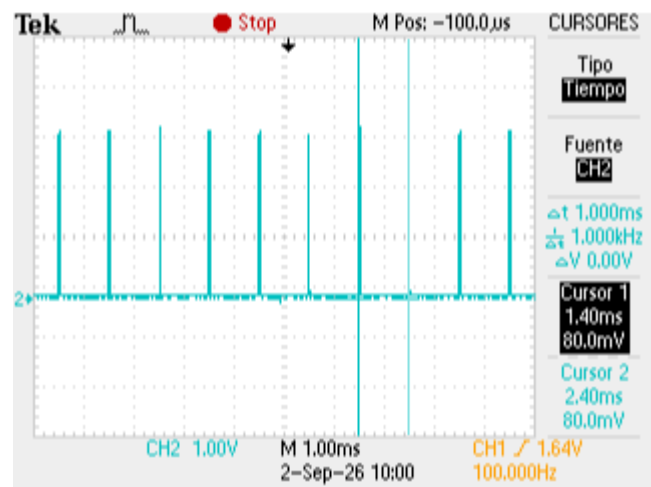
TDS 2012C - 9:55:48 a. m. 2/09/2026
Figure 14.Periodo Muestra 1



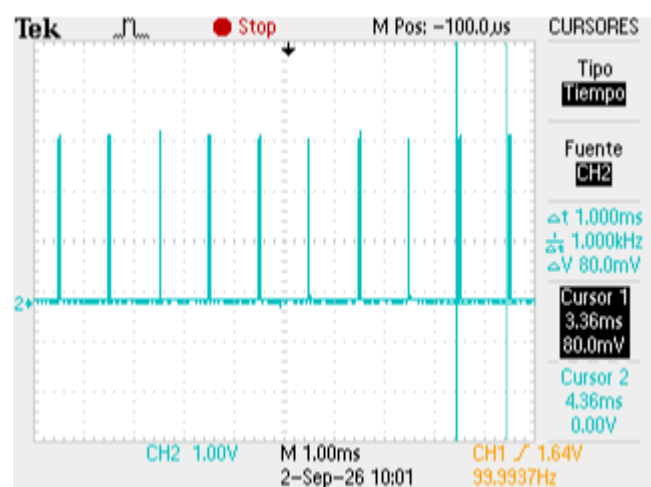
TDS 2012C - 10:10:48 a. m. 2/09/2026
Figure 15.Periodo Muestra 2



TDS 2012C - 10:13:25 a. m. 2/09/2026
Figure 16.Periodo Muestra 3

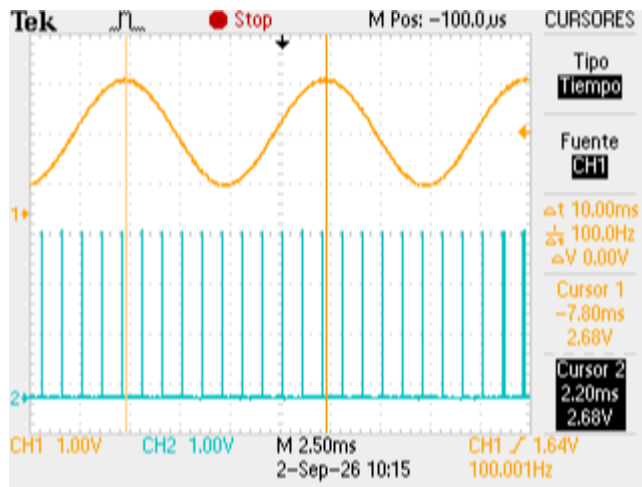


TDS 2012C - 10:11:23 a. m. 2/09/2026
Figure 17. Periodo Muestra 4

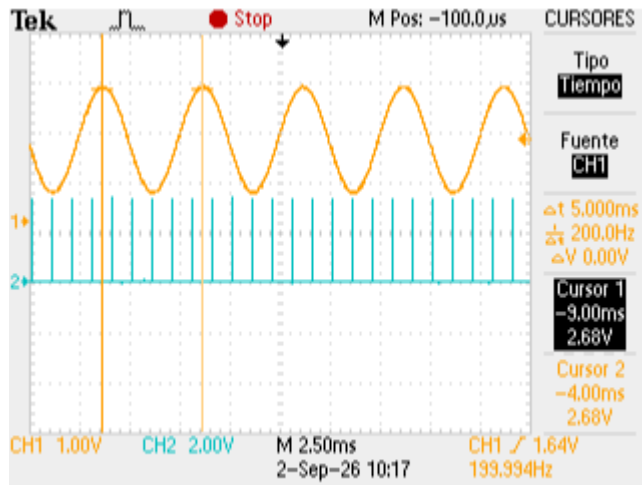


TDS 2012C - 10:12:01 a. m. 2/09/2026
Figure 18.Periodo Muestra 5

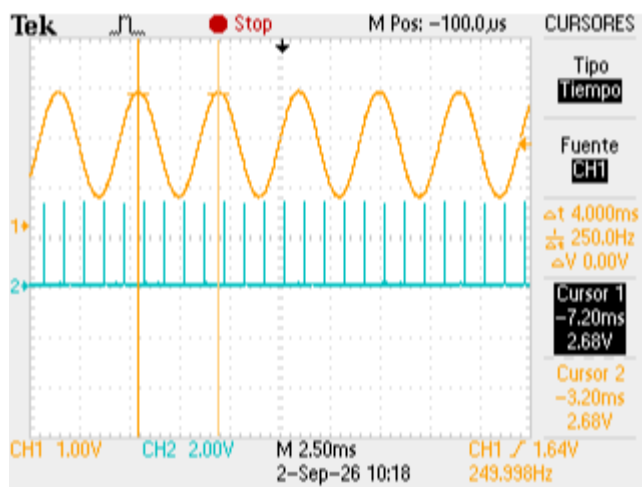
B. ANEXO B Variación Frecuencias



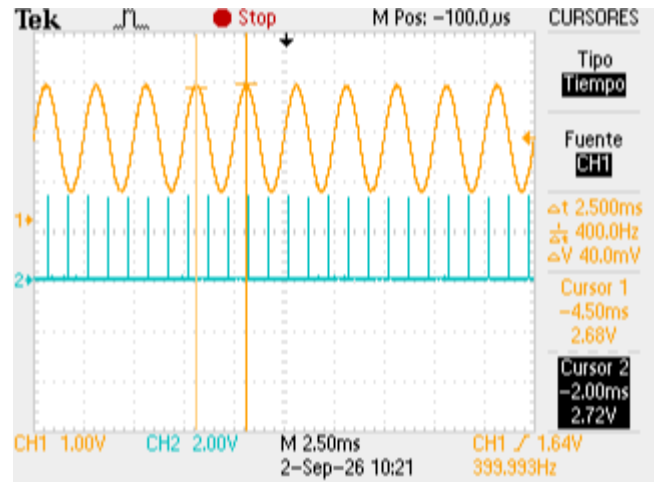
TDS 2012C - 10:26:44 a. m. 2/09/2026
Figure 19. Variación 100 Hz



TDS 2012C - 10:28:29 a. m. 2/09/2026
Figure 20. Variación 200Hz



TDS 2012C - 10:29:54 a. m. 2/09/2026
Figure 21. Variación 250 Hz



TDS 2012C - 10:32:43 a. m. 2/09/2026
Figure 22. Variación 400 Hz

ANEXO C Código sampling 1

```
# sampling 1.py
# Basic ADC sampling example
# Digital Communications
# Telecommunications Engineering Program
# Universidad Militar Nueva Granada - UMNG
# Raspberry Pi Pico 2 W
# jose.rugeles@unimilitar.edu.co
```

```
import machine
import utime
import array
```

```
# -----
# Hardware configuration
# -----
ADC_PIN = 26
SAMPLE_MARKER_PIN = 15
VREF = 3.3
```

```
adc = machine.ADC(ADC_PIN)
sample_marker = machine.Pin(SAMPLE_MARKER_PIN,
machine.Pin.OUT)
sample_marker.value(0)
```

```
# -----
# Sampling parameters
# -----
FS_TARGET = 1000 # Desired sampling frequency [Hz]
SAMPLE_PERIOD_US = int(1_000_000 / FS_TARGET)
NUM_SAMPLES = 10000
```

```
# Output file.
# IMPORTANT: The file is written AFTER the acquisition so
that
# filesystem operations do not disturb the sampling interval.
FILENAME = "datos.csv"
```

```
# Compact buffers:
# times_us -> 32-bit timestamps
# raw_samples -> 16-bit ADC values returned by read_u16()
times_us = array.array("I")
raw_samples = array.array("H")

print("-----")
print("Basic ADC sampling")
print("-----")
print("ADC input      : GP{}".format(ADC_PIN))
print("Sampling marker      : GP{}".format(SAMPLE_MARKER_PIN))
print("Target Fs      : {} Hz".format(FS_TARGET))
print("Target sampling time      : {} us".format(SAMPLE_PERIOD_US))
print("Number of samples      : {}".format(NUM_SAMPLES))
print("Starting acquisition...")

# -----
# Data acquisition
# -----
# Each sample is scheduled relative to the same initial time.
# This avoids accumulating the execution time of the loop from
# sample to sample, as would happen with a simple sleep() after
# every ADC reading.
#
# The rising edge on GP15 marks the instant immediately
# before
# the ADC reading. Connect GP15 to an oscilloscope or logic
# analyzer to measure the real sampling frequency
# experimentally.
# -----
start = utime.ticks_us()

for i in range(NUM_SAMPLES):
    target = utime.ticks_add(start, i * SAMPLE_PERIOD_US)

    # Wait until the programmed sampling instant.
    while utime.ticks_diff(target, utime.ticks_us()) > 0:
        pass

    # Sampling marker:
    # rising edge immediately before the ADC reading.
    sample_marker.value(1)

    t_sample = utime.ticks_us()
    raw_value = adc.read_u16()

    sample_marker.value(0)

    # Store data in RAM.
    # File writing is intentionally postponed until the end.
    times_us.append(utime.ticks_diff(t_sample, start))
    raw_samples.append(raw_value)

print("Acquisition completed.")

# -----

# Estimate the real sampling frequency from recorded
# timestamps
# -----
if NUM_SAMPLES > 1:
    elapsed_us = times_us[-1] - times_us[0]

    mean_period_us = elapsed_us / (NUM_SAMPLES - 1)
    fs_software = 1_000_000 / mean_period_us

    min_period_us = times_us[1] - times_us[0]
    max_period_us = min_period_us

    for i in range(2, NUM_SAMPLES):
        dt = times_us[i] - times_us[i - 1]

        if dt < min_period_us:
            min_period_us = dt

        if dt > max_period_us:
            max_period_us = dt

    print("-----")
    print("Timing results")
    print("-----")
    print("Target Fs      : {:.2f} Hz".format(FS_TARGET))
    print("Software-estimated Fs      : {:.2f} Hz".format(fs_software))
    print("Mean sample period      : {:.2f} us".format(mean_period_us))
    print("Minimum period      : {} us".format(min_period_us))
    print("Maximum period      : {} us".format(max_period_us))
    print("")
    print("Measure GP{} with the oscilloscope to obtain Fs_osc.".format(SAMPLE_MARKER_PIN))

# -----
# Save data AFTER acquisition
# -----
# CSV columns:
# Sample -> sample number
# Time_us -> sampling instant relative to the start
# Raw_u16 -> MicroPython ADC reading scaled to 0..65535
# Voltage_V -> approximate voltage using VREF = 3.3 V
# -----
conversion_factor = VREF / 65535

with open(FILENAME, "w") as f:
    f.write("Sample,Time_us,Raw_u16,Voltage_V\n")

    for i in range(NUM_SAMPLES):
        voltage = raw_samples[i] * conversion_factor

        f.write("{} , {} , {} , {:.6f}\n".format(
            i,
            times_us[i],
```

```
raw_samples[i],
voltage
))
```

```
print("-----")
print("Data saved in: {}".format(FILENAME))
print("Program finished.")
```

ANEXO D Código Sampling 2

```
#
=====
=====

# sampling_2.py
# Raspberry Pi Pico 2 W (RP2350) - MicroPython
# Statistical characterization of a DC input using the ADC
# Digital Communications
# Telecommunications Engineering Program
# Universidad Militar Nueva Granada - UMNG
# Raspberry Pi Pico 2 W
# jose.rugeles@unimilitar.edu.co

# Purpose:
# - Acquire N DC measurements
# - Calculate mean, sample variance, sample standard
deviation
# - Calculate minimum, maximum, and range
# - Build a histogram using approximate 12-bit ADC codes
# - Save raw samples and histogram as CSV files
#
#
=====
=====

from machine import ADC
from array import array
import math
import gc

try:
    import os
except ImportError:
    os = None

ADC_PIN = 26
VREF = 3.3
NUM_SAMPLES = 10000
BUFFER_SAMPLES = 100

adc = ADC(ADC_PIN)
LSB_12_V = VREF / 4095.0

def free_space_bytes():
    if os is None:
        return None
    try:
        stat = os.statvfs("/")
        return stat[0] * stat[3]
```

```
except Exception:
    return None
```

```
print()
print("-----")
print("ADC statistical characterization")
print("Raspberry Pi Pico 2 W")
print("-----")
print("ADC input : GP{}".format(ADC_PIN))
print("Number samples : {}".format(NUM_SAMPLES))
print("Assumed VREF : {:.3f} V".format(VREF))
print("Ideal 12-bit LSB: {:.3f} mV".format(LSB_12_V *
1000))
print()

test_id = input("Test number (1-5): ").strip()
if test_id not in ("1", "2", "3", "4", "5"):
    print("Invalid test number. Using test_1.")
    test_id = "1"

reference_text = input(
    "DC voltage measured with DMM [V] (Enter to skip): "
).strip()

reference_voltage = None
if reference_text:
    try:
        reference_voltage = float(reference_text)
    except ValueError:
        print("Invalid reference voltage. DMM comparison will be
skipped.")
        reference_voltage = None

sample_filename = "samples_test_{}.csv".format(test_id)
hist_filename = "histogram_test_{}.csv".format(test_id)

free_bytes = free_space_bytes()
if free_bytes is not None:
    print("Free filesystem space: {:.1f} kB".format(free_bytes /
1024))

print()
print("Files to be created:")
print(" {}".format(sample_filename))
print(" {}".format(hist_filename))
print()

# Welford statistical variables
n = 0
mean_v = 0.0
M2 = 0.0
min_v = None
max_v = None
min_code12 = 4095
max_code12 = 0
count_code_0 = 0
count_code_4095 = 0
```



```
# 4096 nominal 12-bit bins; 16-bit counts are enough for 10000
samples.
histogram = array("H", [0] * 4096)
gc.collect()
```

```
print("Starting acquisition...")
```

```
try:
    with open(sample_filename, "w") as sample_file:
        # Compact file: students convert Raw_u16 to voltage in
        MATLAB/Python.
```

```
    sample_file.write("Sample,Raw_u16\n")
    buffer = []
```

```
    for i in range(NUM_SAMPLES):
        raw_u16 = adc.read_u16()
        voltage = raw_u16 * VREF / 65535.0
        code12 = raw_u16 >> 4
```

```
    # Welford online statistics
    n += 1
    delta = voltage - mean_v
    mean_v += delta / n
    delta2 = voltage - mean_v
    M2 += delta * delta2
```

```
    if min_v is None or voltage < min_v:
        min_v = voltage
    if max_v is None or voltage > max_v:
        max_v = voltage
```

```
    if code12 < min_code12:
        min_code12 = code12
    if code12 > max_code12:
        max_code12 = code12
```

```
    histogram[code12] += 1
```

```
    if code12 == 0:
        count_code_0 += 1
    if code12 == 4095:
        count_code_4095 += 1
```

```
    buffer.append("{}{}\n".format(i, raw_u16))
```

```
    if len(buffer) >= BUFFER_SAMPLES:
        sample_file.write("".join(buffer))
        buffer = []
```

```
    if (i + 1) % 1000 == 0:
        print("{} / {} samples".format(i + 1,
NUM_SAMPLES))
```

```
    if buffer:
        sample_file.write("".join(buffer))
```

```
except OSError as exc:
```

```
    print()
    print("ERROR while writing sample file:", exc)
    print("Delete old files from the Pico filesystem and run the
test again.")
    raise
```

```
if n > 1:
    variance_v = M2 / (n - 1)
    std_v = math.sqrt(variance_v)
else:
    variance_v = 0.0
    std_v = 0.0
```

```
range_v = max_v - min_v
std_mv = std_v * 1000.0
std_lsb = std_v / LSB_12_V
```

```
# Histogram mode
mode_code12 = 0
mode_count = histogram[0]
for code in range(1, 4096):
    if histogram[code] > mode_count:
        mode_code12 = code
        mode_count = histogram[code]
```

```
mode_voltage = mode_code12 * LSB_12_V
```

```
try:
    with open(hist_filename, "w") as hist_file:
        hist_file.write("Code12,Count,Voltage_equiv_V\n")
        for code in range(4096):
            count = histogram[code]
            if count > 0:
                voltage_equiv = code * LSB_12_V
                hist_file.write(
                    "{}{}\n".format(code, count,
voltage_equiv)
                )
except OSError as exc:
    print()
    print("ERROR while writing histogram file:", exc)
    print("Delete old files from the Pico filesystem and run the
test again.")
    raise
```

```
print()
print("-----")
print("Statistical results")
print("-----")
print("N : {}".format(n))
print("Mean V_bar : {:.6f} V".format(mean_v))
print("Sample variance s^2 : {:.10e} V^2".format(variance_v))
print("Sample std. deviation s : {:.6f} V".format(std_v))
print("Sample std. deviation s : {:.3f} mV".format(std_mv))
print("Std. deviation : {:.3f} LSB".format(std_lsb))
print("Minimum V_min : {:.6f} V".format(min_v))
print("Maximum V_max : {:.6f} V".format(max_v))
```

```
print("Range R          : {:.6f} V".format(range_v))
print("Minimum nominal code : {}".format(min_code12))
print("Maximum nominal code  : {}".format(max_code12))
print("Mode nominal code     : {}".format(mode_code12))
print("Mode count            : {}".format(mode_count))
print("Mode equivalent voltage : {:.6f} V".format(mode_voltage))
print("Code 0 occurrences     : {}".format(count_code_0))
print("Code 4095 occurrences  : {}".format(count_code_4095))
```

```
if reference_voltage is not None:
    error_v = mean_v - reference_voltage
    error_mv = error_v * 1000.0
```

```
print()
print("DMM reference          : {:.6f} V".format(reference_voltage))
print("Mean error ADC - DMM    : {:.6f} V".format(error_v))
print("Mean error ADC - DMM    : {:.6f} mV".format(error_mv))
```

```
if reference_voltage != 0:
    error_percent = 100.0 * error_v / reference_voltage
    print("Relative mean error      : {:.3f} %".format(error_percent))
```

```
print()
print("-----")
print("Files saved")
print("-----")
print(sample_filename)
print(hist_filename)
print()
print("Copy both files to the computer before the next test.")
print("Delete old test files from the Pico if storage space is limited.")
print("-----")
```

ANEXO E Medición referencia multímetro



Figure 23. Verificación Voltaje Test 1

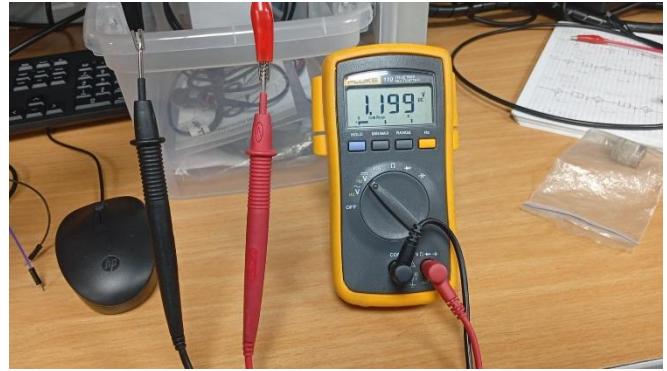


Figure 24. Verificación Voltaje Test 2



Figure 25. Verificación Voltaje Test 3



Figure 26. Verificación Voltaje Test 4



Figure 27. Verificación Voltaje Test 5

ANEXO F Código Python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
VREF = 3.3
# Color asignado a cada ensayo
COLORES = {
    1: "tab:blue",
    2: "tab:orange",
    3: "tab:green",
    4: "tab:red",
    5: "tab:purple"
}
TESTS = {
    1: {"vdmm": 0.599, "soft_mean": 0.597400, "soft_std":
14.451},
    2: {"vdmm": 1.199, "soft_mean": 1.197824, "soft_std":
15.377},
    3: {"vdmm": 1.798, "soft_mean": 1.801162, "soft_std":
15.878},
    4: {"vdmm": 2.399, "soft_mean": 2.404208, "soft_std":
15.542},
    5: {"vdmm": 2.999, "soft_mean": 3.006148, "soft_std":
15.332},
}
def procesar_test(test_id, vdmm, soft_mean=None,
soft_std=None):
    samples_file = f"samples_test_{test_id}.csv"
    hist_file = f"histogram_test_{test_id}.csv"
    color = COLORES[test_id]
    # 1. CARGA DE LOS DATOS
    df = pd.read_csv(samples_file)
    df["Raw_u16"] = df["Raw_u16"].astype(int)
    # Conversión de la lectura de 16 bits a voltaje
    df["Voltage_V"] = df["Raw_u16"] * VREF / 65535.0
    # Recuperación del código nominal de 12 bits
    df["Code12"] = (df["Raw_u16"] // 16).astype(int)
    # 2. CÁLCULO DE PARÁMETROS ESTADÍSTICOS
    v_mean = df["Voltage_V"].mean()
    v_std = df["Voltage_V"].std(ddof=1)
    v_min = df["Voltage_V"].min()
    v_max = df["Voltage_V"].max()
    v_range = v_max - v_min
    error_v = v_mean - vdmm
    error_pct = 100 * error_v / vdmm if vdmm != 0 else np.nan
    print(f"\n===== TEST {test_id} =====")
    print(f"VDMM : {vdmm:.4f} V")
    print(f"Media ADC : {v_mean:.4f} V")
    print(f"Desviación estándar: {v_std*1000:.3f} mV")
    print(f"Mínimo : {v_min:.4f} V")
    print(f"Máximo : {v_max:.4f} V")
    print(f"Rango : {v_range*1000:.3f} mV")
    print(f>Error : {error_pct:.3f} %")
    # 3. COMPARACIÓN CON SAMPLING_2.PY
    if soft_mean is not None:
        diferencia_media = (v_mean - soft_mean) * 1000
        print(f"Media sampling_2.py: {soft_mean:.4f} V")
        print(f"Diferencia media : {diferencia_media:.4f} mV")
    if soft_std is not None:
        diferencia_std = (v_std * 1000) - soft_std
        print(f"s sampling_2.py : {soft_std:.3f} mV")
        print(f"Diferencia s : {diferencia_std:.4f} mV")
    # 4. GRÁFICA DE VOLTAJE VS NÚMERO DE
    MUESTRA
    plt.figure(figsize=(8, 4))
    plt.plot(
        df["Sample"],
        df["Voltage_V"],
        linewidth=0.7,
        color=color
    )
    plt.xlabel("Número de muestra")
    plt.ylabel("Voltaje (V)")
    plt.title(f"Test {test_id}: lecturas del ADC")
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(
        f"test{test_id}_lecturas_vs_muestra.png",
        dpi=150
    )
    plt.show()
    # 5. HISTOGRAMA DE VOLTAJES
    plt.figure(figsize=(6, 4))
    plt.hist(
        df["Voltage_V"],
        bins=30,
        edgecolor="black",
        color=color
    )
    plt.xlabel("Voltaje (V)")
    plt.ylabel("Frecuencia")
    plt.title(f"Test {test_id}: histograma de mediciones")
    plt.tight_layout()
    plt.savefig(
        f"test{test_id}_histograma_voltaje.png",
        dpi=150
    )
    plt.show()
    # 6. HISTOGRAMA DE CÓDIGOS DE 12 BITS
    hist_df = pd.read_csv(hist_file)
    plt.figure(figsize=(6, 4))
    plt.bar(
        hist_df["Code12"],
        hist_df["Count"],
        width=1.0,
        color=color,
        edgecolor="none" # Al eliminar los bordes de cada
        barra, el color del test se muestra limpio
    )
    plt.xlabel("Código ADC de 12 bits")
    plt.ylabel("Frecuencia")
    plt.title(f"Test {test_id}: histograma de códigos ADC")
```

```
plt.tight_layout()
plt.savefig(
    f'test{test_id}_histograma_codigo12.png',
    dpi=150
)
plt.show()
# 7. RESULTADOS
return {
    "Test": test_id,
    "VDMm_V": vdm,
    "V_ADC_V": v_mean,
    "s_mV": v_std * 1000,
    "Error_mV": error_v * 1000,
    "Error_pct": error_pct
}
# PROCESAR LOS CINCO TEST
if __name__ == "__main__":
    resultados = []
    for test_id, cfg in TESTS.items():
        resultado = procesar_test(
            test_id,
            cfg["vdm"],
            cfg["soft_mean"],
            cfg["soft_std"]
        )
        resultados.append(resultado)
# TABLA
tabla = pd.DataFrame(resultados)
print("\n===== TABLA RESUMEN =====")
print(tabla.to_string(index=False))
tabla.to_csv(
    "tabla_resumen_parte2.csv",
    index=False
)
print("\nArchivo generado:")
print("tabla_resumen_parte2.csv")
```

ANEXO G Resultados estadísticos de consola

```
-----
Statistical results
-----
N : 10000
Mean V_bar : 0.597400 V
Sample variance s^2 : 2.0882466000e-04 V^2
Sample std. deviation s : 0.014451 V
Sample std. deviation s : 14.451 mV
Std. deviation : 17.932 LSB
Minimum V_min : 0.458531 V
Maximum V_max : 0.707535 V
Range R : 0.249004 V
Minimum nominal code : 569
Maximum nominal code : 878
Mode nominal code : 739
Mode count : 449
Mode equivalent voltage : 0.595531 V
Code 0 occurrences : 0
Code 4095 occurrences : 0

DMM reference : 0.600000 V
Mean error ADC - DMM : -0.002600 V
Mean error ADC - DMM : -2.600 mV
Relative mean error : -0.433 %
-----
```

Figure 28. Resultados estadísticos de consola — Test 1

```
-----
Statistical results
-----
N : 10000
Mean V_bar : 1.197824 V
Sample variance s^2 : 2.3646011600e-04 V^2
Sample std. deviation s : 0.015377 V
Sample std. deviation s : 15.377 mV
Std. deviation : 19.082 LSB
Minimum V_min : 1.072607 V
Maximum V_max : 1.349810 V
Range R : 0.277203 V
Minimum nominal code : 1331
Maximum nominal code : 1675
Mode nominal code : 1485
Mode count : 489
Mode equivalent voltage : 1.196703 V
Code 0 occurrences : 0
Code 4095 occurrences : 0

DMM reference : 1.200000 V
Mean error ADC - DMM : -0.002177 V
Mean error ADC - DMM : -2.177 mV
Relative mean error : -0.181 %
-----
```

Figure 29. Resultados estadísticos de consola — Test 2


```
-----
Statistical results
-----
N                : 10000
Mean V_bar       : 1.801162 V
Sample variance s^2 : 2.5211510000e-04 V^2
Sample std. deviation s : 0.015878 V
Sample std. deviation s : 15.878 mV
Std. deviation   : 19.703 LSB
Minimum V_min    : 1.589146 V
Maximum V_max    : 1.955024 V
Range R          : 0.365878 V
Minimum nominal code : 1972
Maximum nominal code : 2426
Mode nominal code  : 2237
Mode count       : 456
Mode equivalent voltage : 1.802711 V
Code 0 occurrences : 0
Code 4095 occurrences : 0

DMM reference    : 1.800000 V
Mean error ADC - DMM : +0.001162 V
Mean error ADC - DMM : +1.162 mV
Relative mean error : +0.065 %
-----
```

Figure 30.Resultados estadísticos de consola — Test 3

```
-----
Statistical results
-----
N                : 10000
Mean V_bar       : 2.404208 V
Sample variance s^2 : 2.4154800000e-04 V^2
Sample std. deviation s : 0.015542 V
Sample std. deviation s : 15.542 mV
Std. deviation   : 19.286 LSB
Minimum V_min    : 2.250758 V
Maximum V_max    : 2.543319 V
Range R          : 0.292561 V
Minimum nominal code : 2793
Maximum nominal code : 3156
Mode nominal code  : 2982
Mode count       : 458
Mode equivalent voltage : 2.403077 V
Code 0 occurrences : 0
Code 4095 occurrences : 0

DMM reference    : 2.400000 V
Mean error ADC - DMM : +0.004208 V
Mean error ADC - DMM : +4.208 mV
Relative mean error : +0.175 %
-----
```

Figure 31.Resultados estadísticos de consola — Test 4

```
-----
Statistical results
-----
N                : 10000
Mean V_bar       : 3.006148 V
Sample variance s^2 : 2.3506284000e-04 V^2
Sample std. deviation s : 0.015332 V
Sample std. deviation s : 15.332 mV
Std. deviation   : 19.025 LSB
Minimum V_min    : 2.899528 V
Maximum V_max    : 3.151756 V
Range R          : 0.252227 V
Minimum nominal code : 3598
Maximum nominal code : 3911
Mode nominal code  : 3728
Mode count       : 441
Mode equivalent voltage : 3.004249 V
Code 0 occurrences : 0
Code 4095 occurrences : 0

DMM reference    : 3.000000 V
Mean error ADC - DMM : +0.006149 V
Mean error ADC - DMM : +6.149 mV
Relative mean error : +0.205 %
-----
```

Figure 32.Resultados estadísticos de consola — Test 5

ANEXO H

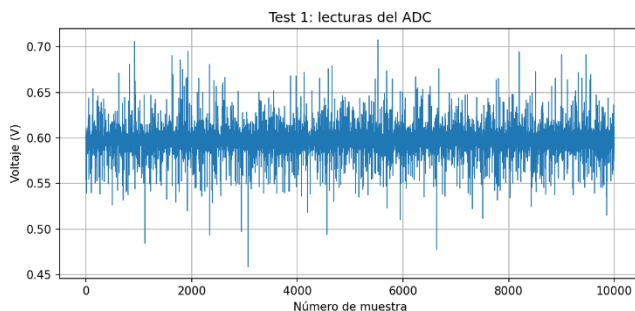


Figure 33.Voltaje vs. número de muestra Test 1

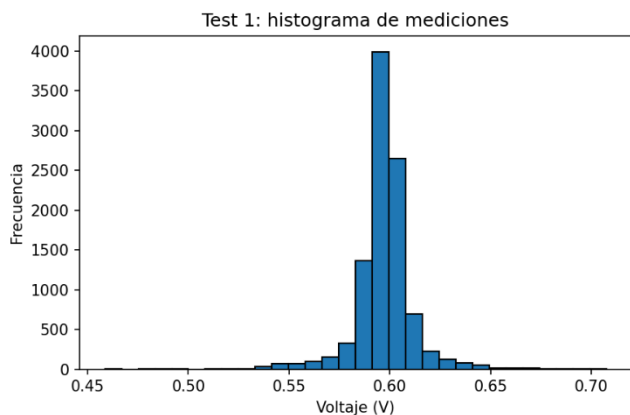


Figure 34. Histograma de las mediciones en voltios para el Test 1.

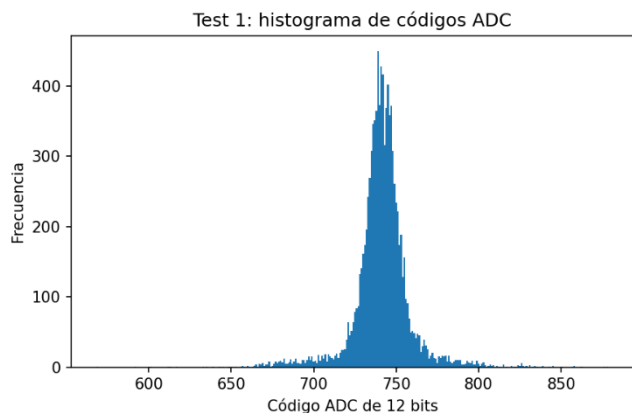


Figure 35. Histograma de códigos ADC de 12 bits (Test 1)

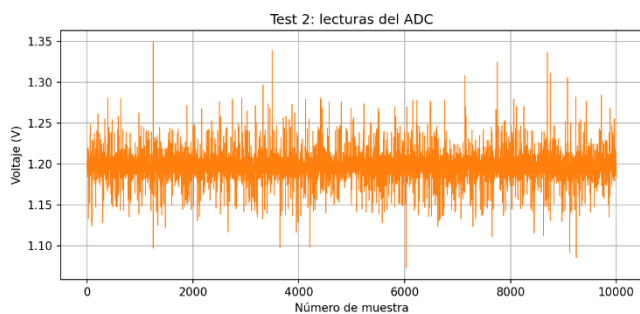


Figure 36. Voltaje vs. número de muestra Test 2.

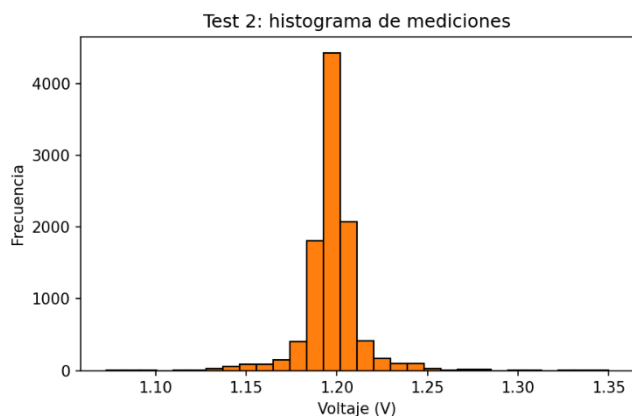


Figure 37. Histograma de las mediciones en voltios Test 2

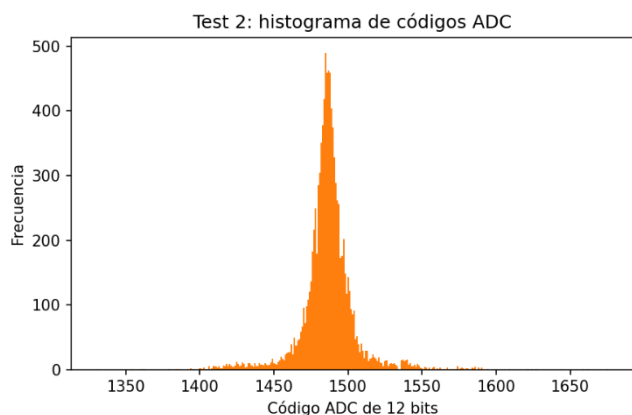


Figure 38. Histograma de códigos ADC de 12 bits Test 2

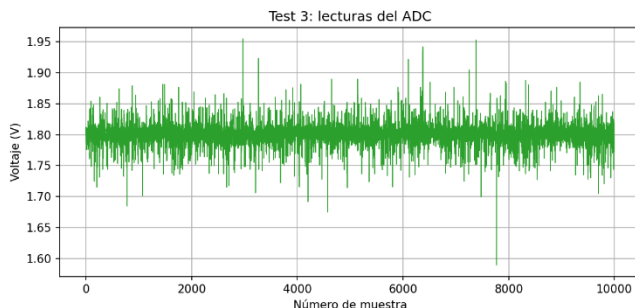


Figure 39. Voltaje vs. número de muestra Test 3

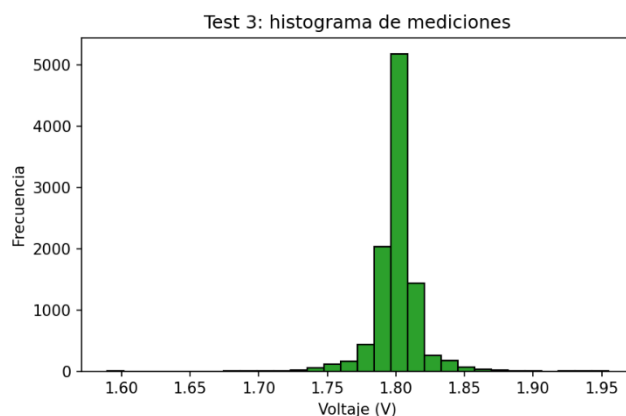


Figure 40. Histograma de las mediciones en voltios Test 3

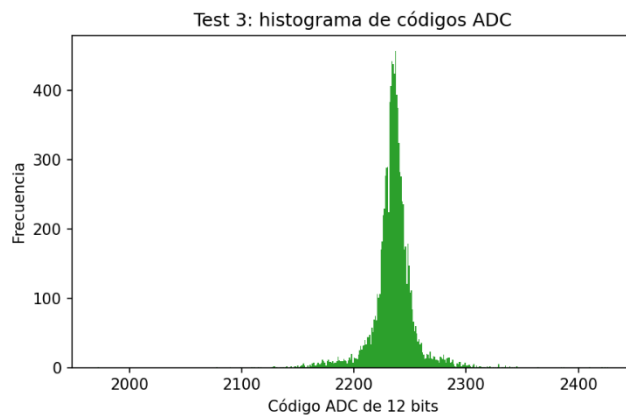


Figure 41. Histograma de códigos ADC de 12 bits Test 3

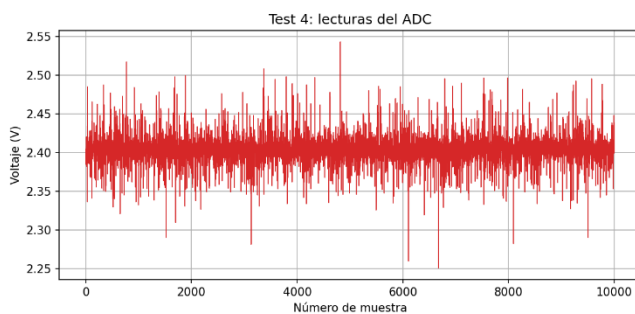


Figure 42. Voltaje vs. número de muestra Test 4

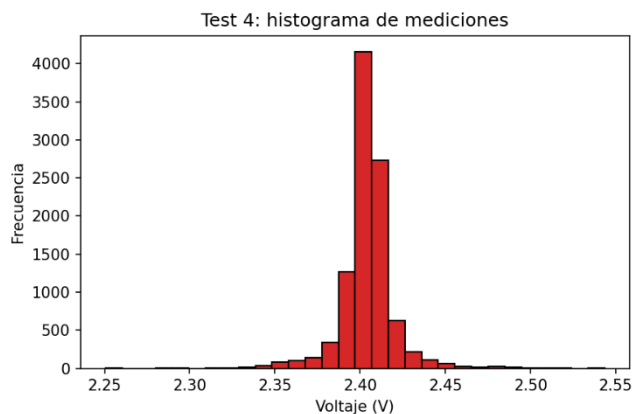


Figure 43. Histograma de las mediciones en voltios Test 4

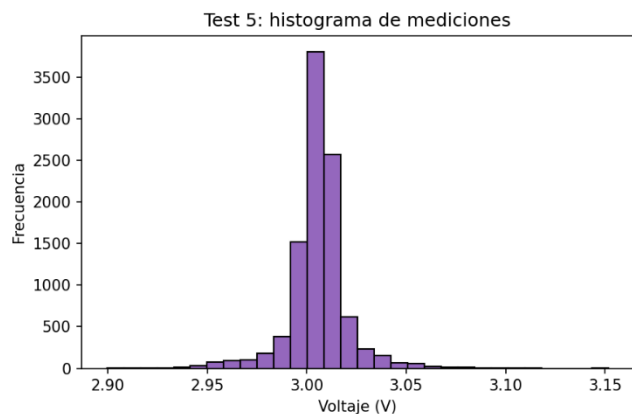


Figure 46. Histograma de las mediciones en voltios Test 5

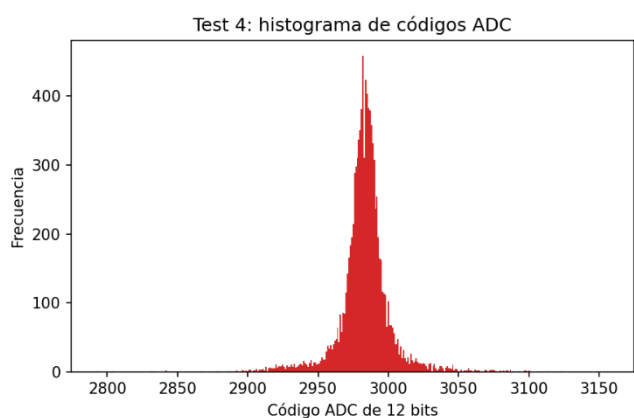


Figure 44. Histograma de códigos ADC de 12 bits Test 4

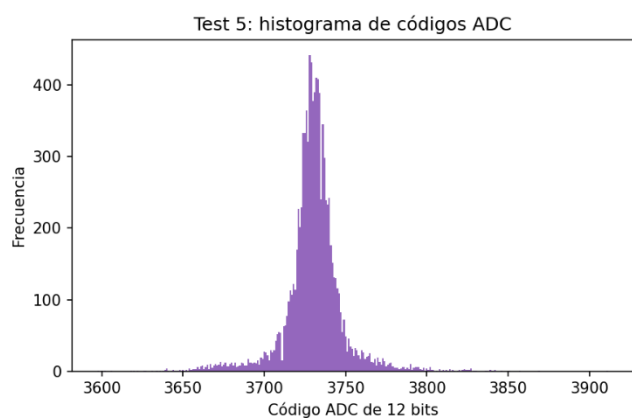


Figure 47. Histograma de códigos ADC de 12 bits Test 5

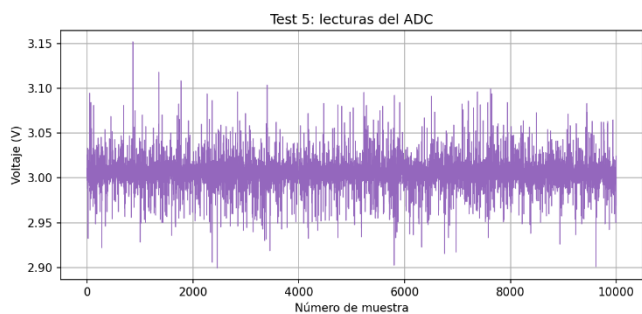


Figure 45. Voltaje vs. número de muestra Test 5